

✓ PART A: Image Classification (CNN).

✓ 1. Imports

Here we import all the necessary libraries for building and training our deep learning models:

- **TensorFlow / Keras** — the main deep learning framework used for building, training, and evaluating models.
- **NumPy** — for numerical operations and array manipulation.
- **Matplotlib / PIL** — for visualizing images and plotting training curves.
- **os / zipfile** — for file system navigation and handling compressed files.
- **ModelCheckpoint / EarlyStopping** — Keras callbacks used to save the best model and stop training early when performance stops improving.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical
import numpy as np
import os
import zipfile
import matplotlib.pyplot as plt
from PIL import Image
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
import matplotlib.pyplot as plt
import random
from PIL import Image
```

✓ 2. Directory

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
test_dir = "/content/drive/MyDrive/AI Assesment/Part 2/Test"
train_dir = "/content/drive/MyDrive/AI Assesment/Part 2/Train"
```

✓ Understanding Data & Visualization

✓ Classes

```
# Get class names (subdirectories)
class_names = sorted(os.listdir(train_dir))
if not class_names:
    print("No class directories found in the train folder!")
else:
    print(f"Found {len(class_names)} classes: {class_names}")

Found 5 classes: ['Banana (1)', 'Cherry (1)', 'Grape (1)', 'Mango (1)', 'Peach (1)']
```

✓ Checking Corrupted Images.

Before training, we scan through every image in the training directory and attempt to open and verify it using PIL. Any image that fails verification is flagged as corrupted and removed. This is an important data cleaning step corrupted images can cause training errors or introduce noise into the learning process.

```
corrupted = []
```

```

for cls in class_names:
    cls_path = os.path.join(train_dir, cls)

    for img_name in os.listdir(cls_path):
        img_path = os.path.join(cls_path, img_name)

        try:
            img = Image.open(img_path)
            img.verify()
        except:
            corrupted.append(img_path)
            os.remove(img_path)
            print(f"Removed corrupted image: {img_path}")

if len(corrupted) == 0:
    print("No Corrupted Images Found.")

```

No Corrupted Images Found.

✓ Class Count Balance

Here we count how many images exist in each fruit class. This is important for identifying **class imbalance** if one class has significantly more images than another, the model may become biased toward predicting the majority class more often.

```

# Dictionary to store class counts
class_counts = {}
for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)
    if os.path.isdir(class_path):
        images = [img for img in os.listdir(class_path) if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
        class_counts[class_name] = len(images) # Count images in each class
# Print Class Balance
print("\nClass Distribution:")
print("=" * 45)
print(f"{'Class Name':<25}{'Valid Image Count':>15}")
print("=" * 45)
for class_name, count in class_counts.items():
    print(f"{'class_name':<25}{count:>15}")
print("=" * 45)

```

```

Class Distribution:
=====
Class Name          Valid Image Count
=====
Banana (1)          1422
Cherry (1)          1222
Grape (1)           1468
Mango (1)            908
Peach (1)           1222
=====

```

✓ Select Random Images for Visualization

```

import random
selected_images = [] # Store image paths
selected_labels = [] # Store corresponding class names
for class_name in class_names:
    class_path = os.path.join(train_dir, class_name)
    if os.path.isdir(class_path):
        images = [img for img in os.listdir(class_path) if img.lower().endswith(('.png', '.jpg', '.jpeg'))]
        if images: # Ensure the class folder is not empty
            selected_img = os.path.join(class_path, random.choice(images))
            selected_images.append(selected_img)
            selected_labels.append(class_name)

```

```

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
# Determine grid size
num_classes = len(selected_images)
cols = (num_classes + 1) // 2 # Determine columns
rows = 2 # Fixed rows for layout

```

```
fig, axes = plt.subplots(rows, cols, figsize=(12, 6))
for i, ax in enumerate(axes.flat):
    if i < num_classes:
        img = mpimg.imread(selected_images[i])
        ax.imshow(img)
        ax.set_title(selected_labels[i])
        ax.axis("off")
    else:
        ax.axis("off") # Hide empty subplots
plt.tight_layout()
plt.show()
```



✓ Data Preprocessing and Generation

Here we define two key configuration values:

- **IMG_SIZE = 128** — all images will be resized to 128×128 pixels to ensure uniform input dimensions for the model.
- **BATCH_SIZE = 32** — images are fed to the model in batches of 32 during training, which balances memory usage and gradient stability.

```
IMG_SIZE = 128
BATCH_SIZE = 32
```

We use `image_dataset_from_directory` to automatically load images from the training folder and split them into **training (80%)** and **validation (20%)** subsets. Keras handles the folder-based label assignment automatically — each subfolder name becomes a class label. The dataset is shuffled with a fixed seed for reproducibility.

```
train_ds = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE
)

val_ds = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
```

```

subset="validation",
seed=42,
image_size=(IMG_SIZE, IMG_SIZE),
batch_size=BATCH_SIZE
)

test_ds = keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    shuffle=False
)

```

```

Found 6242 files belonging to 5 classes.
Using 4994 files for training.
Found 6242 files belonging to 5 classes.
Using 1248 files for validation.
Found 5 files belonging to 5 classes.

```

```

print(f"Contents of {test_dir}:")
files_in_test_dir = os.listdir(test_dir)
if files_in_test_dir:
    print(files_in_test_dir)
else:
    print("The directory is empty or does not exist.")

```

```

Contents of /content/drive/MyDrive/AI Assesment/Part 2/Test:
['Banana', 'Peach', 'Grape', 'Mango', 'Cherry']

```

Data augmentation artificially increases the diversity of our training set by applying random transformations to images during each epoch:

- **RandomFlip** — mirrors images horizontally, simulating fruit viewed from different angles.
- **RandomRotation** — slightly rotates images to handle orientation variance.
- **RandomZoom** — zooms in/out to simulate varying distances.
- **RandomContrast** — adjusts brightness/contrast to simulate different lighting conditions.

These layers are embedded inside the model and are **only active during training**, not during validation or inference.

```

data_augmentation_layers = [
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.1),
]

```

```

import matplotlib.pyplot as plt

for images, labels in train_ds.take(1):
    plt.figure(figsize=(10,5))
    for i in range(6):
        ax = plt.subplot(2,3,i+1)
        augmented = images[i]
        for layer in data_augmentation_layers:
            augmented = layer(augmented)
        plt.imshow(augmented.numpy().astype("uint8"))
        plt.axis("off")

```



✓ Normalization Layer

```
normalization_layer = layers.Rescaling(1./255)
```

✓ Base Line CNN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

num_classes = len(train_ds.class_names)

baseline_model = keras.Sequential([
    layers.Input(shape=(128, 128, 3)),

    *data_augmentation_layers,

    normalization_layer,

    # CNN Layers
    layers.Conv2D(32, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),

    layers.Dense(num_classes, activation='softmax')
])
```

```
baseline_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy', # IMPORTANT CHANGE
    metrics=['accuracy']
)

baseline_model.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param #
random_flip_3 (RandomFlip)	(None, 128, 128, 3)	0
random_rotation_3 (RandomRotation)	(None, 128, 128, 3)	0
random_zoom_3 (RandomZoom)	(None, 128, 128, 3)	0
random_contrast_3 (RandomContrast)	(None, 128, 128, 3)	0
rescaling_3 (Rescaling)	(None, 128, 128, 3)	0
conv2d_13 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_10 (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_14 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_11 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_15 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_12 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten_3 (Flatten)	(None, 25088)	0
dense_13 (Dense)	(None, 128)	3,211,392
dense_14 (Dense)	(None, 64)	8,256
dense_15 (Dense)	(None, 32)	2,080
dense_16 (Dense)	(None, 5)	165

Total params: 3,315,141 (12.65 MB)

Trainable params: 3,315,141 (12.65 MB)

Non-trainable params: 0 (0.00 MB)

```
history_baseline = baseline_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=22
)
```

```
Epoch 1/22
157/157 ━━━━━━━━━━━ 22s 124ms/step - accuracy: 0.9057 - loss: 0.2737 - val_accuracy: 0.9631 - val_loss: 0.0777
Epoch 2/22
157/157 ━━━━━━━━━━━ 19s 122ms/step - accuracy: 0.9886 - loss: 0.0383 - val_accuracy: 0.9872 - val_loss: 0.0315
Epoch 3/22
157/157 ━━━━━━━━━━━ 20s 124ms/step - accuracy: 0.9952 - loss: 0.0164 - val_accuracy: 0.9976 - val_loss: 0.0065
Epoch 4/22
157/157 ━━━━━━━━━━━ 19s 119ms/step - accuracy: 0.9868 - loss: 0.0414 - val_accuracy: 0.9872 - val_loss: 0.0224
Epoch 5/22
157/157 ━━━━━━━━━━━ 20s 127ms/step - accuracy: 0.9974 - loss: 0.0095 - val_accuracy: 1.0000 - val_loss: 0.0020
Epoch 6/22
157/157 ━━━━━━━━━━━ 19s 120ms/step - accuracy: 0.9966 - loss: 0.0108 - val_accuracy: 1.0000 - val_loss: 5.6575e-04
Epoch 7/22
157/157 ━━━━━━━━━━━ 19s 120ms/step - accuracy: 0.9996 - loss: 9.3462e-04 - val_accuracy: 1.0000 - val_loss: 4.0282e-04
Epoch 8/22
157/157 ━━━━━━━━━━━ 19s 122ms/step - accuracy: 0.9998 - loss: 8.9130e-04 - val_accuracy: 1.0000 - val_loss: 3.6649e-05
Epoch 9/22
157/157 ━━━━━━━━━━━ 19s 123ms/step - accuracy: 1.0000 - loss: 1.0832e-04 - val_accuracy: 1.0000 - val_loss: 1.9714e-05
Epoch 10/22
157/157 ━━━━━━━━━━━ 19s 118ms/step - accuracy: 1.0000 - loss: 2.1921e-05 - val_accuracy: 1.0000 - val_loss: 1.2958e-05
Epoch 11/22
157/157 ━━━━━━━━━━━ 19s 119ms/step - accuracy: 1.0000 - loss: 1.7243e-05 - val_accuracy: 1.0000 - val_loss: 9.0222e-06
Epoch 12/22
157/157 ━━━━━━━━━━━ 20s 127ms/step - accuracy: 0.9862 - loss: 0.0461 - val_accuracy: 1.0000 - val_loss: 0.0046
Epoch 13/22
157/157 ━━━━━━━━━━━ 19s 119ms/step - accuracy: 0.9818 - loss: 0.0698 - val_accuracy: 1.0000 - val_loss: 0.0060
Epoch 14/22
157/157 ━━━━━━━━━━━ 19s 121ms/step - accuracy: 0.9966 - loss: 0.0105 - val_accuracy: 1.0000 - val_loss: 0.0025
Epoch 15/22
157/157 ━━━━━━━━━━━ 20s 124ms/step - accuracy: 0.9996 - loss: 0.0011 - val_accuracy: 1.0000 - val_loss: 4.2440e-04
Epoch 16/22
157/157 ━━━━━━━━━━━ 19s 120ms/step - accuracy: 0.9996 - loss: 0.0018 - val_accuracy: 1.0000 - val_loss: 6.6268e-05
Epoch 17/22
157/157 ━━━━━━━━━━━ 20s 130ms/step - accuracy: 1.0000 - loss: 8.9162e-05 - val_accuracy: 1.0000 - val_loss: 1.2751e-04
Epoch 18/22
157/157 ━━━━━━━━━━━ 20s 126ms/step - accuracy: 1.0000 - loss: 1.2793e-04 - val_accuracy: 1.0000 - val_loss: 6.2050e-05
Epoch 19/22
```

```

157/157 ————— 19s 123ms/step - accuracy: 1.0000 - loss: 7.5567e-05 - val_accuracy: 1.0000 - val_loss: 4.9647e-05
Epoch 20/22
157/157 ————— 21s 133ms/step - accuracy: 1.0000 - loss: 5.7601e-05 - val_accuracy: 1.0000 - val_loss: 1.2691e-05
Epoch 21/22
157/157 ————— 19s 121ms/step - accuracy: 1.0000 - loss: 1.3072e-05 - val_accuracy: 1.0000 - val_loss: 1.2522e-05
Epoch 22/22
157/157 ————— 19s 121ms/step - accuracy: 1.0000 - loss: 8.4084e-05 - val_accuracy: 1.0000 - val_loss: 1.1155e-05

```

```

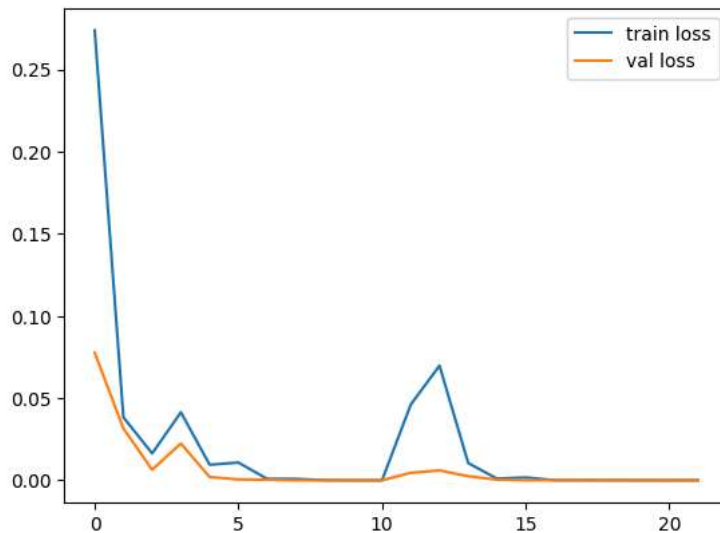
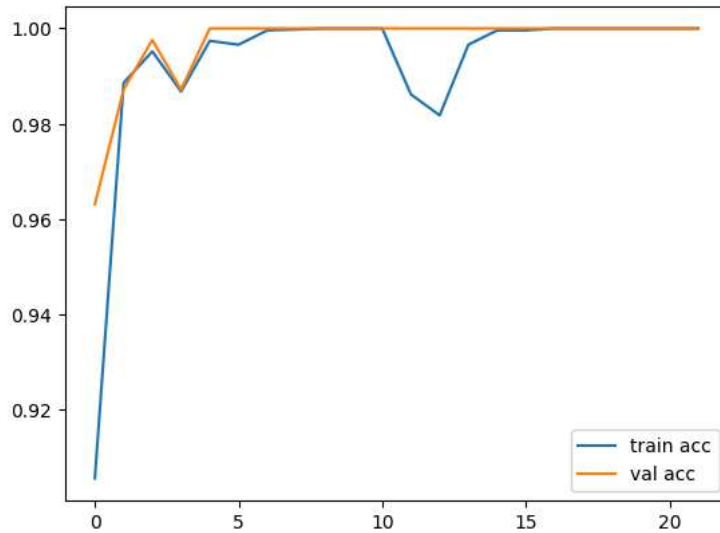
plt.plot(history_baseline.history['accuracy'], label='train acc')
plt.plot(history_baseline.history['val_accuracy'], label='val acc')
plt.legend()
plt.show()

```

```

plt.plot(history_baseline.history['loss'], label='train loss')
plt.plot(history_baseline.history['val_loss'], label='val loss')
plt.legend()
plt.show()

```



Base Line Model Classification Report

```

import numpy as np
from sklearn.metrics import classification_report

y_true = []

for images, labels in test_ds:
    y_true.extend(labels.numpy())

y_true = np.array(y_true)

```

```

pred = baseline_model.predict(test_ds)
y_pred = np.argmax(pred, axis=1)

class_names = test_ds.class_names

print(classification_report(y_true, y_pred, target_names=class_names))

```

```

1/1 ————— 0s 107ms/step

```

	precision	recall	f1-score	support
Banana	1.00	1.00	1.00	1
Cherry	1.00	1.00	1.00	1
Grape	1.00	1.00	1.00	1
Mango	1.00	1.00	1.00	1
Peach	1.00	1.00	1.00	1
accuracy			1.00	5
macro avg	1.00	1.00	1.00	5
weighted avg	1.00	1.00	1.00	5

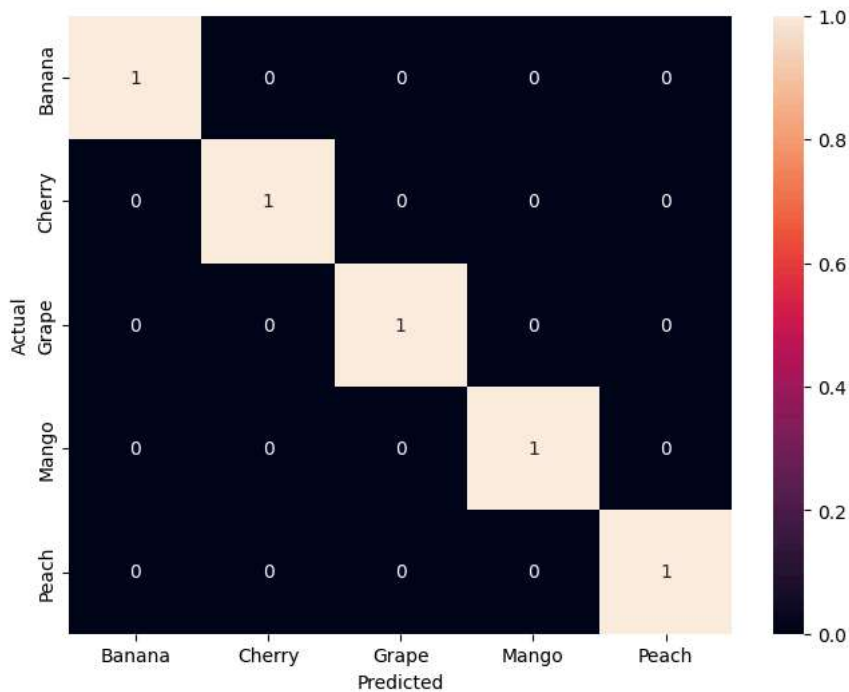
```

from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



✓ Deeper Model

```

from tensorflow import keras
from tensorflow.keras import layers

num_classes = len(train_ds.class_names)

deep_model = keras.Sequential([
    layers.Input(shape=(128, 128, 3)),

    # ✓ Data Augmentation (same as before)
    *data_augmentation_layers,

```



```
#  Normalize
layers.Rescaling(1./255),

#  Block 1
layers.Conv2D(32, (3,3), padding='same', activation='relu'),
layers.BatchNormalization(),
layers.Conv2D(32, (3,3), activation='relu'),
layers.MaxPooling2D(),
layers.Dropout(0.25),

#  Block 2
layers.Conv2D(64, (3,3), padding='same', activation='relu'),
layers.BatchNormalization(),
layers.Conv2D(64, (3,3), activation='relu'),
layers.MaxPooling2D(),
layers.Dropout(0.25),

#  Block 3
layers.Conv2D(128, (3,3), padding='same', activation='relu'),
layers.BatchNormalization(),
layers.Conv2D(128, (3,3), activation='relu'),
layers.MaxPooling2D(),
layers.Dropout(0.3),

#  Extra Depth (important for assignment)
layers.Conv2D(256, (3,3), activation='relu'),
layers.BatchNormalization(),
layers.MaxPooling2D(),
layers.Dropout(0.3),

#  Fully Connected
layers.Flatten(),

layers.Dense(256, activation='relu'),
layers.BatchNormalization(),
layers.Dropout(0.5),

layers.Dense(128, activation='relu'),
layers.Dropout(0.5),

layers.Dense(num_classes, activation='softmax')
])

deep_model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

deep_model.summary()
```

Model: "sequential_5"

Layer (type)	Output Shape	Param #
random_flip_3 (RandomFlip)	(None, 128, 128, 3)	0
random_rotation_3 (RandomRotation)	(None, 128, 128, 3)	0
random_zoom_3 (RandomZoom)	(None, 128, 128, 3)	0
random_contrast_3 (RandomContrast)	(None, 128, 128, 3)	0
rescaling_4 (Rescaling)	(None, 128, 128, 3)	0
conv2d_16 (Conv2D)	(None, 128, 128, 32)	896
batch_normalization_6 (BatchNormalization)	(None, 128, 128, 32)	128
conv2d_17 (Conv2D)	(None, 126, 126, 32)	9,248
max_pooling2d_13 (MaxPooling2D)	(None, 63, 63, 32)	0
dropout_7 (Dropout)	(None, 63, 63, 32)	0
conv2d_18 (Conv2D)	(None, 63, 63, 64)	18,496
batch_normalization_7 (BatchNormalization)	(None, 63, 63, 64)	256
conv2d_19 (Conv2D)	(None, 61, 61, 64)	36,928
max_pooling2d_14 (MaxPooling2D)	(None, 30, 30, 64)	0
dropout_8 (Dropout)	(None, 30, 30, 64)	0
conv2d_20 (Conv2D)	(None, 30, 30, 128)	73,856
batch_normalization_8 (BatchNormalization)	(None, 30, 30, 128)	512
conv2d_21 (Conv2D)	(None, 28, 28, 128)	147,584
max_pooling2d_15 (MaxPooling2D)	(None, 14, 14, 128)	0
dropout_9 (Dropout)	(None, 14, 14, 128)	0
conv2d_22 (Conv2D)	(None, 12, 12, 256)	295,168
batch_normalization_9 (BatchNormalization)	(None, 12, 12, 256)	1,024
max_pooling2d_16 (MaxPooling2D)	(None, 6, 6, 256)	0
dropout_10 (Dropout)	(None, 6, 6, 256)	0
flatten_4 (Flatten)	(None, 9216)	0

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
```

```
lr_scheduler = ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.3,
    patience=3
)
```

```
Max_trainable_params: 1.472 / 5.75 KB
```

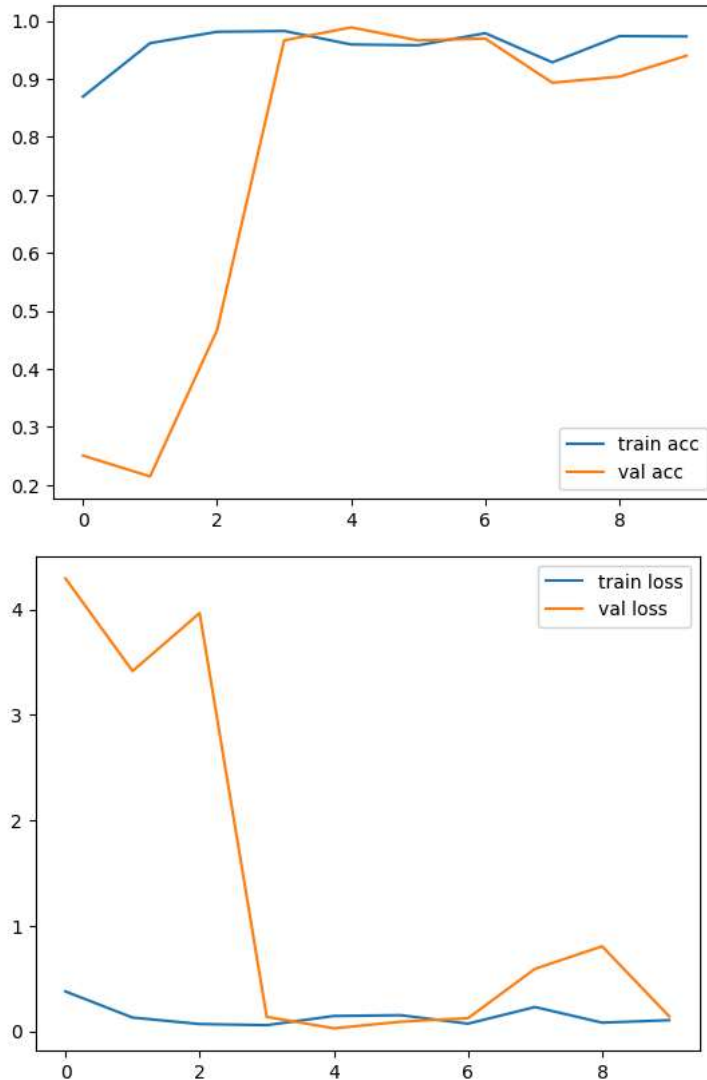
```
history_deep = deep_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=26,
    callbacks=[early_stop]
)
```

Epoch 1/26
157/157 ————— 26s 134ms/step - accuracy: 0.8694 - loss: 0.3780 - val_accuracy: 0.2508 - val_loss: 4.2943

```
Epoch 2/26
157/157 ————— 21s 131ms/step - accuracy: 0.9616 - loss: 0.1299 - val_accuracy: 0.2147 - val_loss: 3.4160
Epoch 3/26
157/157 ————— 20s 125ms/step - accuracy: 0.9812 - loss: 0.0688 - val_accuracy: 0.4671 - val_loss: 3.9655
Epoch 4/26
157/157 ————— 23s 143ms/step - accuracy: 0.9826 - loss: 0.0580 - val_accuracy: 0.9663 - val_loss: 0.1366
Epoch 5/26
157/157 ————— 21s 130ms/step - accuracy: 0.9594 - loss: 0.1451 - val_accuracy: 0.9888 - val_loss: 0.0278
Epoch 6/26
157/157 ————— 22s 137ms/step - accuracy: 0.9579 - loss: 0.1521 - val_accuracy: 0.9663 - val_loss: 0.0910
Epoch 7/26
157/157 ————— 20s 129ms/step - accuracy: 0.9788 - loss: 0.0723 - val_accuracy: 0.9696 - val_loss: 0.1240
Epoch 8/26
157/157 ————— 21s 134ms/step - accuracy: 0.9285 - loss: 0.2299 - val_accuracy: 0.8934 - val_loss: 0.5926
Epoch 9/26
157/157 ————— 22s 142ms/step - accuracy: 0.9738 - loss: 0.0817 - val_accuracy: 0.9038 - val_loss: 0.8058
Epoch 10/26
157/157 ————— 21s 133ms/step - accuracy: 0.9732 - loss: 0.1050 - val_accuracy: 0.9399 - val_loss: 0.1410
```

```
plt.plot(history_deep.history['accuracy'], label='train acc')
plt.plot(history_deep.history['val_accuracy'], label='val acc')
plt.legend()
plt.show()

plt.plot(history_deep.history['loss'], label='train loss')
plt.plot(history_deep.history['val_loss'], label='val loss')
plt.legend()
plt.show()
```



✓ Deeper CNN with Adam Optimizer

```
import numpy as np
from sklearn.metrics import classification_report

y_true = []

for images, labels in test_ds:
    y_true.extend(labels.numpy())

y_true = np.array(y_true)

pred = deep_model.predict(test_ds)
y_pred = np.argmax(pred, axis=1)

class_names = test_ds.class_names

print(classification_report(y_true, y_pred, target_names=class_names))
```

```
1/1 ----- 1s 586ms/step
              precision    recall  f1-score   support

   Banana      1.00      1.00      1.00         1
    Cherry      1.00      1.00      1.00         1
     Grape      1.00      1.00      1.00         1
     Mango      1.00      1.00      1.00         1
      Peach      1.00      1.00      1.00         1

 accuracy              1.00         5
  macro avg           1.00      1.00      1.00         5
 weighted avg           1.00      1.00      1.00         5
```

✓ Deeper CNN with SGD optimizer

```
from tensorflow.keras.optimizers import SGD

deep_model_sgd = keras.models.clone_model(deep_model)

deep_model_sgd.compile(
    optimizer=SGD(learning_rate=0.01),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history_sgd = deep_model_sgd.fit(
    train_ds,
    validation_data=val_ds,
    epochs=20,
    callbacks=[early_stop]
)
```

```
Epoch 1/20
157/157 ----- 23s 129ms/step - accuracy: 0.6856 - loss: 0.8730 - val_accuracy: 0.2059 - val_loss: 4.6973
Epoch 2/20
157/157 ----- 21s 131ms/step - accuracy: 0.8534 - loss: 0.4028 - val_accuracy: 0.2508 - val_loss: 5.2135
Epoch 3/20
157/157 ----- 20s 125ms/step - accuracy: 0.9353 - loss: 0.1924 - val_accuracy: 0.2508 - val_loss: 8.2683
Epoch 4/20
157/157 ----- 20s 126ms/step - accuracy: 0.9586 - loss: 0.1347 - val_accuracy: 0.3029 - val_loss: 10.6761
Epoch 5/20
157/157 ----- 21s 131ms/step - accuracy: 0.9742 - loss: 0.0929 - val_accuracy: 0.5288 - val_loss: 14.6209
```

✓ Part B: Fine Tuning

Transfer Learning with MobilNetV2

```
IMG_SIZE = 224

train_ds = keras.utils.image_dataset_from_directory(
    train_dir,
    validation_split=0.2,
    subset="training",
    seed=42,
    image_size=(IMG_SIZE, IMG_SIZE),
```

```

        batch_size=32
    )

    val_ds = keras.utils.image_dataset_from_directory(
        train_dir,
        validation_split=0.2,
        subset="validation",
        seed=42,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=32
    )

    test_ds = keras.utils.image_dataset_from_directory(
        test_dir,
        image_size=(IMG_SIZE, IMG_SIZE),
        batch_size=32,
        shuffle=False
    )

```

```

Found 6242 files belonging to 5 classes.
Using 4994 files for training.
Found 6242 files belonging to 5 classes.
Using 1248 files for validation.
Found 5 files belonging to 5 classes.

```

```

data_augmentation_layers = [
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.1),
]

```

✓ Loading the Pre trained model

```

base_model = keras.applications.MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False, # ! Remove original classifier
    weights='imagenet'
)

```

```
base_model.trainable = False
```

```

num_classes = len(train_ds.class_names)

model = keras.Sequential([
    layers.Input(shape=(224, 224, 3)),

    *data_augmentation_layers,

    layers.Lambda(keras.applications.mobilenet_v2.preprocess_input),

    base_model,

    layers.GlobalAveragePooling2D(),

    layers.BatchNormalization(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(num_classes, activation='softmax')
])

```

```

model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential_6"

Layer (type)	Output Shape	Param #
random_flip_4 (RandomFlip)	(None, 224, 224, 3)	0
random_rotation_4 (RandomRotation)	(None, 224, 224, 3)	0
random_zoom_4 (RandomZoom)	(None, 224, 224, 3)	0
random_contrast_4 (RandomContrast)	(None, 224, 224, 3)	0
lambda_1 (Lambda)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization_11 (BatchNormalization)	(None, 1280)	5,120
dense_20 (Dense)	(None, 128)	163,968
dropout_13 (Dropout)	(None, 128)	0
dense_21 (Dense)	(None, 5)	645

Total params: 2,427,717 (9.26 MB)

Trainable params: 167,173 (653.02 KB)

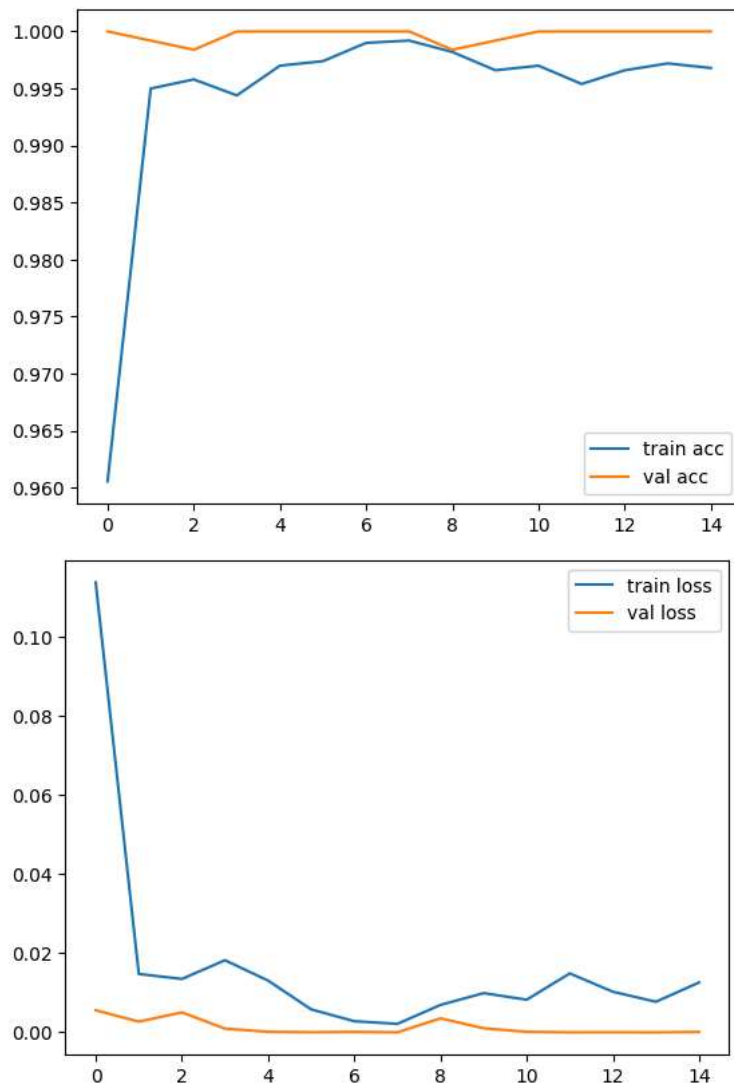
Non-trainable params: 2,260,544 (8.62 MB)

```
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15
)
```

```
Epoch 1/15
157/157 ————— 34s 176ms/step - accuracy: 0.9606 - loss: 0.1139 - val_accuracy: 1.0000 - val_loss: 0.0056
Epoch 2/15
157/157 ————— 26s 168ms/step - accuracy: 0.9950 - loss: 0.0148 - val_accuracy: 0.9992 - val_loss: 0.0027
Epoch 3/15
157/157 ————— 37s 143ms/step - accuracy: 0.9958 - loss: 0.0135 - val_accuracy: 0.9984 - val_loss: 0.0051
Epoch 4/15
157/157 ————— 40s 138ms/step - accuracy: 0.9944 - loss: 0.0183 - val_accuracy: 1.0000 - val_loss: 9.4891e-04
Epoch 5/15
157/157 ————— 21s 131ms/step - accuracy: 0.9970 - loss: 0.0131 - val_accuracy: 1.0000 - val_loss: 1.3599e-04
Epoch 6/15
157/157 ————— 22s 139ms/step - accuracy: 0.9974 - loss: 0.0058 - val_accuracy: 1.0000 - val_loss: 2.2096e-05
Epoch 7/15
157/157 ————— 21s 131ms/step - accuracy: 0.9990 - loss: 0.0028 - val_accuracy: 1.0000 - val_loss: 1.1019e-04
Epoch 8/15
157/157 ————— 41s 131ms/step - accuracy: 0.9992 - loss: 0.0022 - val_accuracy: 1.0000 - val_loss: 1.2677e-05
Epoch 9/15
157/157 ————— 41s 131ms/step - accuracy: 0.9982 - loss: 0.0069 - val_accuracy: 0.9984 - val_loss: 0.0035
Epoch 10/15
157/157 ————— 22s 141ms/step - accuracy: 0.9966 - loss: 0.0099 - val_accuracy: 0.9992 - val_loss: 0.0010
Epoch 11/15
157/157 ————— 21s 132ms/step - accuracy: 0.9970 - loss: 0.0083 - val_accuracy: 1.0000 - val_loss: 1.2722e-04
Epoch 12/15
157/157 ————— 22s 139ms/step - accuracy: 0.9954 - loss: 0.0149 - val_accuracy: 1.0000 - val_loss: 6.3533e-06
Epoch 13/15
157/157 ————— 21s 131ms/step - accuracy: 0.9966 - loss: 0.0103 - val_accuracy: 1.0000 - val_loss: 1.4501e-05
Epoch 14/15
157/157 ————— 22s 143ms/step - accuracy: 0.9972 - loss: 0.0077 - val_accuracy: 1.0000 - val_loss: 4.1019e-06
Epoch 15/15
157/157 ————— 22s 141ms/step - accuracy: 0.9968 - loss: 0.0126 - val_accuracy: 1.0000 - val_loss: 1.0342e-04
```

```
plt.plot(history.history['accuracy'], label='train acc')
plt.plot(history.history['val_accuracy'], label='val acc')
plt.legend()
plt.show()

plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='val loss')
plt.legend()
plt.show()
```



```
import numpy as np
from sklearn.metrics import classification_report

y_true = []

for images, labels in test_ds:
    y_true.extend(labels.numpy())

y_true = np.array(y_true)

pred = model.predict(test_ds)
y_pred = np.argmax(pred, axis=1)

class_names = test_ds.class_names

print(classification_report(y_true, y_pred, target_names=class_names))
```

```
1/1 |-----| 2s 2s/step
      precision    recall  f1-score   support

   Banana         1.00      1.00      1.00         1
   Cherry         1.00      1.00      1.00         1
   Grape          1.00      1.00      1.00         1
   Mango          1.00      1.00      1.00         1
   Peach          1.00      1.00      1.00         1

 accuracy              1.00         5
  macro avg           1.00      1.00      1.00         5
 weighted avg           1.00      1.00      1.00         5
```

```
base_model.trainable = True
```

```
# Freeze lower layers, train only top layers
for layer in base_model.layers[:-30]:
    layer.trainable = False
```

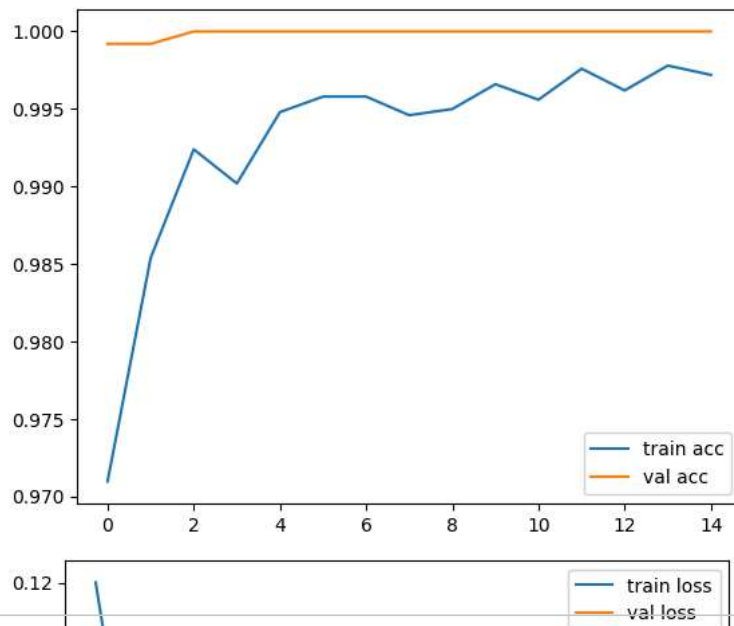
```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-5), # ⚠️ smaller LR
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
history_fine = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=15
)
```

```
Epoch 1/15
157/157 ————— 34s 154ms/step - accuracy: 0.9710 - loss: 0.1200 - val_accuracy: 0.9992 - val_loss: 0.0049
Epoch 2/15
157/157 ————— 22s 138ms/step - accuracy: 0.9854 - loss: 0.0479 - val_accuracy: 0.9992 - val_loss: 0.0038
Epoch 3/15
157/157 ————— 25s 157ms/step - accuracy: 0.9924 - loss: 0.0339 - val_accuracy: 1.0000 - val_loss: 3.8627e-04
Epoch 4/15
157/157 ————— 39s 146ms/step - accuracy: 0.9902 - loss: 0.0372 - val_accuracy: 1.0000 - val_loss: 1.6778e-06
Epoch 5/15
157/157 ————— 41s 145ms/step - accuracy: 0.9948 - loss: 0.0208 - val_accuracy: 1.0000 - val_loss: 8.4577e-06
Epoch 6/15
157/157 ————— 22s 139ms/step - accuracy: 0.9958 - loss: 0.0121 - val_accuracy: 1.0000 - val_loss: 2.1587e-06
Epoch 7/15
157/157 ————— 25s 156ms/step - accuracy: 0.9958 - loss: 0.0177 - val_accuracy: 1.0000 - val_loss: 5.1397e-06
Epoch 8/15
157/157 ————— 23s 146ms/step - accuracy: 0.9946 - loss: 0.0194 - val_accuracy: 1.0000 - val_loss: 2.2183e-06
Epoch 9/15
157/157 ————— 22s 140ms/step - accuracy: 0.9950 - loss: 0.0192 - val_accuracy: 1.0000 - val_loss: 2.7319e-08
Epoch 10/15
157/157 ————— 23s 148ms/step - accuracy: 0.9966 - loss: 0.0109 - val_accuracy: 1.0000 - val_loss: 6.1037e-08
Epoch 11/15
157/157 ————— 22s 141ms/step - accuracy: 0.9956 - loss: 0.0131 - val_accuracy: 1.0000 - val_loss: 1.4901e-08
Epoch 12/15
157/157 ————— 41s 138ms/step - accuracy: 0.9976 - loss: 0.0077 - val_accuracy: 1.0000 - val_loss: 1.2895e-08
Epoch 13/15
157/157 ————— 42s 143ms/step - accuracy: 0.9962 - loss: 0.0134 - val_accuracy: 1.0000 - val_loss: 8.3388e-08
Epoch 14/15
157/157 ————— 22s 141ms/step - accuracy: 0.9978 - loss: 0.0080 - val_accuracy: 1.0000 - val_loss: 1.4003e-07
Epoch 15/15
157/157 ————— 23s 145ms/step - accuracy: 0.9972 - loss: 0.0104 - val_accuracy: 1.0000 - val_loss: 5.1007e-08
```

```
plt.plot(history_fine.history['accuracy'], label='train acc')
plt.plot(history_fine.history['val_accuracy'], label='val acc')
plt.legend()
plt.show()
```

```
plt.plot(history_fine.history['loss'], label='train loss')
plt.plot(history_fine.history['val_loss'], label='val loss')
plt.legend()
plt.show()
```

```
import numpy as np
from sklearn.metrics import classification_report

y_true = []

for images, labels in test_ds:
```